WUOLAH

ALG-17-18-RESUELTO.pdf

JUMIACOR

Algorítmica

2º Grado en Ingeniería Informática

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada



Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Esta es tu señal para
abrir tu Cuenta NoCuenta de ING
y llevarte 5€ por la cara.
Quiero 5€





Abre tu Cuenta con el código WUOLAH5







do your thing

Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial
E.T.S.I. Informática y de Telecomunicación, C/Periodista Daniel Saucedo Aranda s/n - 18071 - Granada (España)

Grado en Informática

Algorítmica

Curso 2017/2018. Convocatoria ordinaria de junio
15 de junio de 2018

1. (2 puntos) Calcular el orden de eficiencia en notación $O(\cdot)$ del algoritmo cuya expresión de tiempo es:

$$T(n) = T(\frac{n}{2}) + 2T(\frac{n}{4}) + n$$

con $T(1) = 1$, $T(2) = 1$.

2. (2 puntos) Una fábrica de pinturas necesita envasar, para satisfacer un pedido, la cantidad de V litros de cierta pintura. Para ello dispone de una determinada cantidad de envases de n tipos distintos, e_1, e_2,..., e_n. La capacidad de cada envase de tipo e_i es de l_i litros, y tiene un coste (el envase) de c_i euros. Además, de cada tipo de envase disponemos de u_i unidades.

   Se desea determinar la forma de satisfacer el pedido utilizando la combinación de envases que genere el mínimo coste. Nótese que es posible que en la solución propuesta algunos envases no queden completamente llenos. Diseñad un algoritmo lo más eficiente posible para calcular la solución óptima. Aplicadlo al siguiente caso: $n = 3$, $V = 10$, y

capacidad l_i	número u_i	coste c_i
3	3	5
5	3	9
7	2	12

3. (2 puntos) Se va a celebrar una cena de gala a la que asistirán n invitados. Todos se van a sentar alrededor de una única gran mesa rectangular, de forma que cada invitado tendrá sentados junto a él a otros dos comensales (uno a su izquierda y otro a su derecha). En función de las características de cada invitado (por ejemplo categoría o puesto, lugar de procedencia,...) existen unas normas de protocolo que indican el nivel de conveniencia de que dos invitados se sienten en lugares contiguos (supondremos que dicho nivel es un número entero entre 0 y 100). El nivel de conveniencia total de una asignación de invitados a su puesto en la mesa es la suma de todos los niveles de conveniencia de cada invitado con cada uno de los dos invitados sentados a su lado. Se desea sentar a los invitados de forma que el nivel de conveniencia total sea lo mayor posible.

   Diseñad un algoritmo de exploración de grafos para encontrar la solución óptima del problema. Especificad, además del algoritmo, la representación del problema, la representación de la solución, las restricciones explícitas e implícitas, así como las posibles cotas a utilizar.

4. (2 puntos) Explicar detalladamente el algoritmo de ordenación rápida *quicksort*. Analizar la importancia de la elección del pivote en el algoritmo. Aplicarlo para ordenar los siguientes elementos (especificar claramente los pasos seguidos por el algoritmo): $3, 7, 1, 2, 0, 5, 6, 4$.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

1. (2 puntos) Calcular el orden de eficiencia en notación $O(\cdot)$ del algoritmo cuya expresión de tiempo es:

$$T(n) = T(\tfrac{n}{2}) + 2T(\tfrac{n}{4}) + n$$

con $T(1) = 1$, $T(2) = 1$.

$T(n) = T(n/2) + 2T(n/4) + n$ / $n = 2^k \quad k = \log_2(n)$

$T(2^k) = T(2^{k-1}) + 2T(2^{k-2}) + 2^k$

$T(k) = T(k-1) + 2T(k-2) + 2^k$

$T(k) - T(k-1) - 2T(k-2) = 2^k$

Parte No Homogénea

$b^k \cdot p(k)$

$(x-b)^{gr(p)+1}$

❶ $x^2 - x - 2 = (x-2)^1$ → igual

$$\frac{+1 \pm \sqrt{1-4\cdot 1\cdot(-2)}}{2} = \frac{-1 \pm \sqrt{9}}{2} \begin{cases} x = 2 \\ x = -1 \end{cases}$$ } inversos

❷ $(x-2)^2 (x+1) = 0$ → se cambia signo

- Ecuación General = $(C_1 \cdot k + C_2)\cdot(2^k) + C_3(-1)^k$

- Ecuación General = $C_1 k \cdot 2^k + C_2 \cdot 2^k + C_3(-1)^k$

$C_1 \cdot n \cdot \log(n) + C_2 \cdot n + C_3(-1)^{\log_2(n)}$

$O(n \cdot \log(n))$

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

ING BANK NV se encuentra adherido al Sistema de Garantía de Depósitos Holandés con una garantía de hasta 100.000 euros por depositante. Consulta más información en ing.es

1/6
Este número es indicativo del riesgo del producto, siendo 1/6 indicativo de menor riesgo y 6/6 de mayor riesgo.

Si ya tuviste sufi con tanto estudio...
Te dejamos este espacio
para desahogarte.

Pinta, arranca,
llora... tú decides ;)



¿Te sientes más liberado?
Sigue siéndolo con la **Cuenta NoCuenta:**
libre de comisiones*, y de lloraditas.

Y ahora con 5€ por la cara.

¡Quiero una de esas!

*TIN 0 % y TAE 0 %.

do your thing

Algorítmica



Banco de apuntes de la

WUOLAH

Comparte estos flyers en tu clase y consigue más dinero y recompensas

1. Imprime esta hoja
2. Recorta por la mitad
3. Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes
4. Llévate dinero por cada descarga de los documentos descargados a través de tu QR

1. (2 puntos) Calcular el orden de eficiencia en notación $O(\cdot)$ del algoritmo cuya expresión de tiempo es:

$$T(n) = T(\frac{n}{2}) + 2T(\frac{n}{4}) + n$$

con $T(1) = 1$, $T(2) = 1$.

$n = 2^k \quad k = \log_2(n)$

$T(2^k) = T(2^{k-1}) + 2T(2^{k-2}) + 2^k$

$x^2 - x - 2 \quad (x-2)$

$\frac{1 \pm \sqrt{1-4\cdot1\cdot(-2)}}{2} = \frac{1 \pm 3}{2}$ $\quad x = 2, \quad x = -1$

$(x-2)^2(x+1)$

$(C_1 \cdot k + C_2)\, 2^k + C_3 \cdot 1^k$

$C_1 \cdot k \cdot 2^k + C_2 \cdot 2^k + C_3 \cdot 1^k$

$C_1 \cdot \log_2(n) \cdot n + C_2 \cdot n + C_3 \Rightarrow O(n \cdot \log(n))$

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

2. (2 puntos) Una fábrica de pinturas necesita envasar, para satisfacer un pedido, la cantidad de V litros de cierta pintura. Para ello dispone de una determinada cantidad de envases de n tipos distintos, e_1, e_2,..., e_n. La capacidad de cada envase de tipo e_i es de l_i litros, y tiene un coste (el envase) de c_i euros. Además, de cada tipo de envase disponemos de u_i unidades.

Se desea determinar la forma de satisfacer el pedido utilizando la combinación de envases que genere el mínimo coste. Nótese que es posible que en la solución propuesta algunos envases no queden completamente llenos. Diseñad un algoritmo lo más eficiente posible para calcular la solución óptima. Aplicadlo al siguiente caso: $n = 3$, $V = 10$, y

tipo	capacidad l_i	número u_i	coste c_i
1	3	3	5
2	5	3	9
3	7	2	12

} Datos

{ Prog. Dinámica: Eficiente, Solución Óptima
↓ Backtracking, B&B ✗
↓ Greedy ✗

* Parecido al de la mochila, pero con objetos repetidos y minimizando el "beneficio"

1 REPRESENTACIÓN DEL PROBLEMA

* Secuencia de decisiones: Decido o no usar el bote de pintura P[i+1, j+1] => donde "i" es el tipo de bote a valorar y "j" es la cantidad de "i" litros.

2 OPTIMALIDAD DE BELMAN

Suponemos que $P[i,j] = \alpha$ es la solución óptima, y que a su vez $P[i-1, j-l[i]\cdot k] = \beta < \alpha - c[i]\cdot k$;

Entonces si sumamos $\beta + c[i]\cdot k$, obtenemos una solución para el problema $P[i,j]$ que además cumple $\alpha > \beta + c[i]$. Por tanto α no sería la solución óptima al problema (contradicción)

Así demostramos que sí se trata de una solución óptima y que se compone de subestructuras suboptimales.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

Esta es tu señal para
abrir tu Cuenta NoCuenta de ING
y **llevarte 5€ por la cara.**



Quiero 5€







☑ Abre tu Cuenta con el código **WUOLAH5**







do your thing

3) DEFINICIÓN RECURSIVA

$P[i,j]$ => mínimo coste para satisfacer el pedido con los primeros "i" envoses

$$P[i,j] = \begin{cases} \forall k \in \{0,1,2,\dots u[i]\} \ \min\left(P[i-1, j-k*l_i] + k*c_i\right) \\ \text{(desde "0" hasta "nº unidades")} \\ \text{iteración} \end{cases}$$

CASOS BASE

$$\left.\begin{array}{l} \forall i \in \{0,1,2\dots v\} \ P[i,0]=0 \\ \forall j \in \{0,1,2\dots e\} \ P[0,j]=+\infty \end{array}\right\}$$

4) PSEUDOCÓDIGO

```
for (int i=0; i<n; i++){
    P[i,0] = 0;
for (int j=1; j<v; j++){
    P[0,j] = INF;                          } CASOS BASE

for (int i=1; i<n; i++){
    for (int j=1; j<v; j++) {
        min = +∞;
        for (int k=0; k<u[i]; k++) { //Probamos con distinto nº de
            if( ([P[i-1,j-k*l[i]]+k·c[i]) < min)   //unidades de un mismo envase
                { min = P[i-1,j-k*l[i]] + k·c[i];
                  P[i,j] = P[i-1,j-k*l[i]] + k*c[i]; }
        }
    }
}
return P[n,v];
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

5/6 HACER TABLA Y RECUPERAR SOLUCIÓN

if P[i-1, j] // No se utiliza el bote, por lo que pasamos a comprobar P[i-1, j]

else // Sí se utiliza el bote

P[i , j - l[c]]

CONSTRUCCIÓN DE LA TABLA

b → / c	0	1	2	3	4	5	6	7	8	9	10
0 →	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
3/5	0	∞	∞	5	10	10	10	15	15	15	20
5/9	0	∞	∞	5	9	9	10	15	15	15	18
7/12	0	∞	∞	5	9	9	10	12	15	15	17

{ 1 bote de 7 y 1 bote de 3 }

3 - 3
5 - 3
7 2
} Cantidades

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

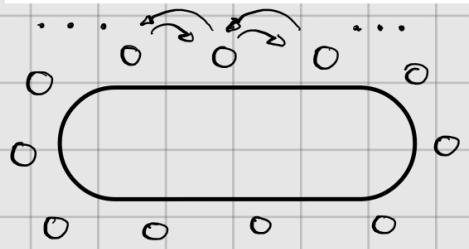
WUOLAH

Las descargas sin publicidad se realizan con las coins

3. (2 puntos) Se va a celebrar una cena de gala a la que asistirán n invitados. Todos se van a sentar alrededor de una única gran mesa rectangular, de forma que cada invitado tendrá sentados junto a él a otros dos comensales (uno a su izquierda y otro a su derecha). En función de las características de cada invitado (por ejemplo categoría o puesto, lugar de procedencia,...) existen unas normas de protocolo que indican el nivel de conveniencia de que dos invitados se sienten en lugares contiguos (supondremos que dicho nivel es un número entero entre 0 y 100). El nivel de conveniencia total de una asignación de invitados a su puesto en la mesa es la suma de todos los niveles de conveniencia de cada invitado con cada uno de los dos invitados sentados a su lado. Se desea sentar a los invitados de forma que el nivel de conveniencia total sea lo mayor posible.

Backtracking / B&B

Diseñad un algoritmo de exploración de grafos para encontrar la solución óptima del problema. Especificad, además del algoritmo, la representación del problema, la representación de la solución, las restricciones explícitas e implícitas, así como las posibles cotas a utilizar.



Toma de decisiones => Vector en el que la posición es la silla y el contenido es la persona

x[i] => i: silla x[i]: persona



Para ver la conveniencia ("V" Matriz) => V[x[i]][x[i-1]] * V[x[i]][x[i+1]]

↓ lo dice el teacher

Restricciones explícitas (Valores del vector solución)

$x[i] \in \{0, n-1\}$ -> tiene que ser de las personas disponibles

Restricciones implícitas (Componentes factibles en la solución)

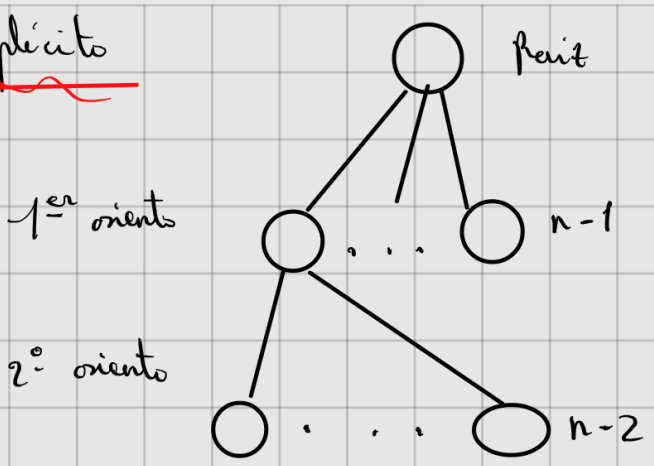
x[i] != x[j] "No puedo sentar a la misma persona en 2 asientos"

WUOLAH

Las descargas sin publicidad se realizan con las coins

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Árbol implícito



Va disminuyendo el factor de ramificación [Una persona y un asiento menos]

* Siempre fijamos a la misma persona la primera silla para no tener soluciones "diferentes" pero equivalentes `x[0] = 0`

Función factibilidad

```
bool factible (int persona) {
    bool = true;
    for (int i = 0; i < j ; i++)
        if (x[persona] == x[i])
            bool = false;

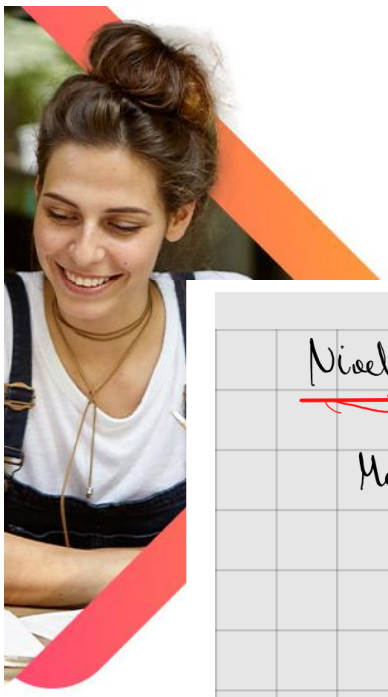
    return bool;
}
```

También se podría hacer un vector auxiliar de booleanos de tamaño "personas" (j), para ver si está sentado ya o no.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins



Curso General o Intensivo,
sácate el título en 3 meses.

Centro preparador.
Exámenes de Cambridge.

TU ACADEMIA de inglés online

Presencial u Online
-10% o -20%
¡Infórmate!

si te apuntas con meses de antelación!!

imllazubia.com

Nivel de conveniencia actual:

Matriz de conveniencia $(i \times j)$ (personas × personas)

	0	1	...
0	10	8	...
1	2	9	...
.	.	.	
.	.	.	
.	.	.	

Representación Problema ⊗ Diagonal de matriz nula ya que no se puede sentar a una persona al lado de sí mismo

Cota global - mejor nivel de conveniencia encontrado hasta ahora en una solución

Cota local optimista - Calculamos, aunque no sea factible, la persona más conveniente para cada persona, de las que quedan por escoger, y se lo sumamos a la conveniencia global actual.

Función poda - Según cota local y global

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

ALGORITMO

```
void backSentar (vector &p, vector &x, vector &sillas, int sentados, float conse){
    float mejor; if (mejor==0) int sillas = sillas.size();
    if (sentados == sillas) // Una solución
        if (mejor==0 || conse < mejor) mejor = conse; //Act Cota Global
        print "Solucion encontrada", x, conse; //Solución y conveniencia global
    }
    else {
        candidato = p[ sillas - (sillas - sentados)];
        p.delete (candidato);
        sentados ++;
          c  = calculo (conseAct (conse, x, candidato)
        conse += c
        if (cotaLocal (conse, p, sillas) < mejor && mejor != 0) //Poda
            backSentar (p, x, sillas, sentados, conse); // según cota local y global
        }
        p.push_back (candidato);
        sentados--;
        conse -= c;
    }
}
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

ALGORITMO MEJORADO

```
int backSentar (vector&x, vector&p, int sentados, conse ) {
   float mejor = 0; int total; if(mejor==0); total = p .size();
   if( sentados == total){
      if(mejor != 0 && conse > mejor)
         mejor = conse;
         print mejor;
      }
   }
   else {
      if (sentados == 0) { //Fijo el primer comensal a la primera silla
         x[0] = 0;
         p.delete[0];
         sentados ++;
         backSentar (x, p, sentados, conse);
      }
      else {// Creamos a los hijos y evaluamos
         for (int i = 1; i < total ; i ++ ){
            if ( !x.exists(i)) { //Factible
               p.delete[i];
               x[sentados] = i;
               sentados ++;
               aux = calcularConse(x, conse);
               conse += aux;
               if ( cotaLocal (x, conse, p) < mejor && mejor != 0){
                  backSentar (x, p, sentados, conse);
               sentados --;
               p.push_back(i);
               x[sentados] = 0
            } conse -= aux
         } } }
      Return mejor
   }
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

4. (2 puntos) Explicar detalladamente el algoritmo de ordenación rápida *quicksort*. Analizar la importancia de la elección del pivote en el algoritmo. Aplicarlo para ordenar los siguientes elementos (especificar claramente los pasos seguidos por el algoritmo): $3, 7, 1, 2, 0, 5, 6, 4$.

```
void quicksort ( vector<tipo> a, int low, int high) {
  if ( low < high ) { //igual que en mergesort para parar
    int pivote = pivotar (a, low, high);
    quicksort ( a, low, pivote-1);
    quicksort ( a, pivote+1, high);
  }

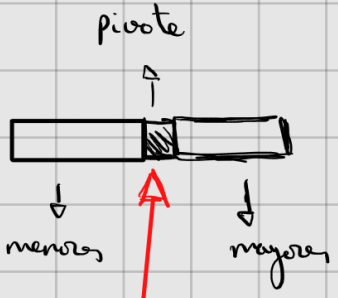
int pivotar ( a, low, high ) {
  int pivote = a[high]; // Se coge un elemento como
  int i = low-1;        // pivote

  for (int j = low; j < high; j++) {
    if ( a[j] < pivote )
      i++;
      swap(a[i], a[j]);
  }
  swap( a[i+1], a[high]); //Colocamos el pivote
  return (i+1);           //en su posición
}
```

1º

2º

3º



Se aumenta el i

Para luego colocar el pivote

Devuelvo el nuevo índice del pivote

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Las descargas sin publicidad se realizan con las coins

facebook NATIONAL GEOGRAPHIC Evernote accenture
Aurgi Telefónica twilio Microsoft
4Geeks academy
¡Estas son las empresas que contratan a nuestros alumnos!
ES POSIBLE.
¿CONVERTIRSE EN FULL STACK DEVELOPER EN 18 SEMANAS?
PIDE MÁS INFO

* En el caso promedio $T(n) = 2n \cdot \log(n) + O(n)$

$O(n \cdot \log(n))$

* En su peor caso (Cuando el pivote seleccionado es el elemento más pequeño o grande de la partición, ya que realiza muchas particiones) es $O(n^2)$

Ejemplo) 3, 7, 1, 2, 0, 5, 6, [4]

3 1 2 0 [4] 5 6 7

3 1 2 [0] 4 5 6 [7]

0 1 2 3 4 5 6 7

0 1 2 3 4 5 6 7

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

5. (2 puntos) Un turista neoyorkino quiere recorrer n países europeos $\{p_1, \ldots, p_n\}$ en un orden prefijado (hay que visitarlos en ese orden) y visitar en cada país p_i una (y sólo una) de entre m_i posibles ciudades prefijadas de ese país. El criterio para determinar su viaje es que el coste total del viaje sea el menor posible (incluyendo el coste de llegar a la ciudad del primer país desde Nueva York y el de volver a Nueva York desde la ciudad del último país). El coste total del viaje es la suma de los costes de viajar de una ciudad a otra. Formalizad el problema y construid un algoritmo que lo resuelva de la forma más eficiente posible, encontrando la solución óptima.

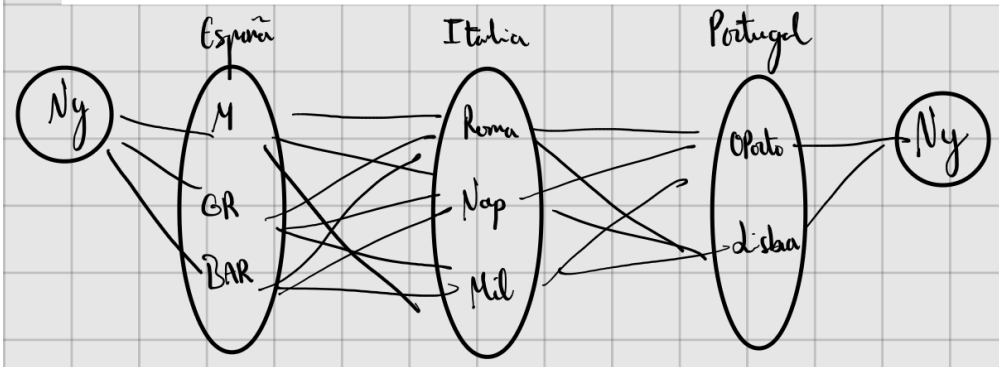
Duración del examen: 2 horas y 30 minutos.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

5. (2 puntos) Un turista neoyorkino quiere recorrer n países europeos $\{p_1, \ldots, p_n\}$ en un orden prefijado (hay que visitarlos en ese orden) y visitar en cada país p_i una (y sólo una) de entre m_i posibles ciudades prefijadas de ese país. El criterio para determinar su viaje es que el coste total del viaje sea el menor posible (incluyendo el coste de llegar a la ciudad del primer país desde Nueva York y el de volver a Nueva York desde la ciudad del último país). El coste total del viaje es la suma de los costes de viajar de una ciudad a otra. Formalizad el problema y construid un algoritmo que lo resuelva de la forma más eficiente posible, encontrando la solución óptima.



Grafo no dirigido => Mínimo camino NY-NY pasando por una ciudad de cada país

Usaremos el algoritmo de Dijkstra:

- CANDIDATOS - nodos pertenecientes al camino mínimo (ciudades)
- SELECCIONADOS - nodos para los que ya se ha calculado el camino mínimo
- F. SOLUCIÓN - Empezar en NY, pasar por "n" países europeos y volver a NY.
- F. OBJETIVO - Calcular el camino mínimo
- F. SELECCIÓN - Selecciona el nodo (ciudad de país) que tenga menor distancia al origen
- F. FACTIBILIDAD - Seleccionar de cada país una sola ciudad (uno nodo por país).

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

```
set<vertices> dijkstra Turista (Grafo G(V,A), nodo raiz) {
    for (int i=0; i<V.size(); i++) { //La raíz es NY
        dist[v] = INF; //Se inicializan los dist y los padres de los vértices
        padre[v] = null;
    }
    set<vertices> S;
    distancia[raiz] = 0;
    priority_queue Q;
    Q.insert(raiz); //Empezamos en NY
    for (int i=0; i<V.size(); i++) Q.insert(vi);
+   bool vuelta_NY = false
    while (!Q.empty() && !vuelta_NY) { //F. Solución
        v = Q.top
        Q.delete(v);
        if (v.pais.visitado == false) { //Factibilidad
*           S.insert(V);
            V' = v.verticesAdyacentes();
            for (int i=0; i<V'.size(); i++) {
*               if (dist[V'[i]] > dist[v] + camino(v, v'[i])){
                    dist(V'[i]) = dist[v] + camino(v, V'[i]);
                    padre(V'[i]) = v;
                }
            }
        }
//F. OBJETIVO     if (v.pais.visitado == true && v.pais == "EEUU") {
                    vuelta_NY = true;
                }
                v.pais.visitado = true; --> Lo último
        }
    }
    return S;
}
```

insertar vértice en Solución y calcula distancias de sus adyacentes

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

ColaCao
UNA PERSONA SENTADA EN SU HABITACIÓN PORQUE TIENE QUE ESTUDIAR. ¿CÓMO SE LLAMA LA PELÍCULA? TU VIDA AHORA MISMO.
COLACAO BATIDOS TE ACOMPAÑA EN ESTO.

5. (2 puntos) Un turista neoyorkino quiere recorrer n países europeos $\{p_1, \ldots, p_n\}$ en un orden prefijado (hay que visitarlos en ese orden) y visitar en cada país p_i una (y sólo una) de entre m_i posibles ciudades prefijadas de ese país. El criterio para determinar su viaje es que el coste total del viaje sea el menor posible (incluyendo el coste de llegar a la ciudad del primer país desde Nueva York y el de volver a Nueva York desde la ciudad del último país). El coste total del viaje es la suma de los costes de viajar de una ciudad a otra. Formalizad el problema y construid un algoritmo que lo resuelva de la forma más eficiente posible, encontrando la solución óptima.

Dijkstra (Camino Mínimo)

- Candidatos - Vértices para los que todavía no se les ha calculado el camino mínimo
- Seleccionados - Vértices a los que sí se les ha calculado el camino mínimo.
- F. Selección - Seleciono el nodo (ciudad) que esté a menos distancia del nodo origen
- F. Factibilidad - No se puede seleccionar más de una ciudad de un mismo país.
- F. Solución - Un nodo distinto a la raíz es NY.
- F. Objetivo - Crear el camino mínimo

```
set<vertices> dijkstraNewYork (Grafo (V, A), vertice raiz) //raiz = NY
{
    for (int i=0; i<V.size(); i++) { //Iniciolito distancias y padres
        dist[vi] = INF;
        padre[vi] = null;
    }
    dist[raiz] = 0;
    set<vertices> S;
    priority_queue Q;
    for (int i=0; i<V.size(); i++)  Q.insert(vi);

* bool vuelta_NY = falso;
  while (!Q.empty() || !vuelta_NY) { // F. Solución
      v = Q.top()
      Q.pop_back();
      if (v.pais.visitado == "falso") // Función Factibilidad
          S.insert(v);
          V' = sacarAdyacentes(v);
          for (int i=0; i<V'.size(); i++)
              if (dist[v'i] > dist[v] + camino(v, v'i))
                  dist[v'i] = dist[v] + camino(v, v'i);
                  padre[v'i] = v;
  }
}
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        if ( a.pais.visitado == true && a.pais == EEUU )
            vuelta_NY = true;
        }
    }
    return S;
}
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins